

TP : Communication, Sécurité et Contrôle d'un Robot Mobile sous ROS 2

Filière Robotique

Mai 2026

Table des matières

1	Exercice 1 : Traitement d'Images et Perception de l'Environnement (Vision)	3
1.1	Idée générale	3
1.2	Travail demandé	3
2	Exercice 2 : Contrôle par Correcteur PD et Gestion Événementielle	4
2.1	Idée générale	4
2.2	Travail demandé	4
3	Exercice 3 : Intégration Système et Communication Inter-Nœuds	5
3.1	Idée générale	5
3.2	Travail demandé	5

Travaux Pratiques : Robotique Mobile & Autonomie

Partie III : Guidage par Vision Artificielle et Intégration Système sous ROS 2

Introduction et Objectifs du TP

Ce troisième volet de travaux pratiques marque une transition importante dans notre étude de l'autonomie des robots mobiles. Alors que la première partie était focalisée sur la perception géométrique 2D et l'évitement d'obstacles à l'aide d'un capteur LiDAR, ce nouveau module introduit l'utilisation d'une **caméra monoculaire** comme capteur principal de navigation.

L'objectif général est de concevoir, d'implémenter et d'analyser un système complet de **suivi de ligne autonome** (type « *Line Follower* » haut de gamme), capable de guider un véhicule robotisé de manière fluide sur une piste sinueuse, tout en faisant preuve de robustesse face aux perturbations du monde réel (changements brusques de luminosité, reflets au sol, perte temporaire de marquage ou intersections complexes).

Pour y parvenir, ce TP adopte une approche système découplée en deux nœuds ROS 2 distincts qui s'exécutent en parallèle :

1. **Le Nœud de Perception** (`vision_line_node`) : Il traite le flux vidéo brut de la caméra en temps réel à l'aide de la bibliothèque de vision par ordinateur **OpenCV**. Il extrait la géométrie de la route sous forme d'une erreur de centrage pixel et d'un tableau d'indicateurs de présence.
2. **Le Nœud de Commande** (`control_line_node`) : Il traduit les données de perception en commandes motrices (`/cmd_vel`) en s'appuyant sur un correcteur **Proportionnel-Dérivé (PD)** couplé à une **Machine à États (FSM)** événementielle pour la gestion des cas critiques (virages en épingle, bifurcations).

Compétences et Concepts Clés Visés

À l'issue de ce TP, vous aurez acquis des compétences concrètes sur :

- **Le traitement d'images appliqué à la robotique** : Segmentation de couleur dans l'espace colorimétrique HSV, fenêtrage géométrique multi-horizons, réduction du bruit par filtres de persistance temporelle (files `deque`) et validation géométrique adaptative.
- **L'automatique décisionnelle** : Modélisation d'un correcteur PD sur flux de données visuelles, optimisation du profil de vitesse linéaire selon l'erreur géométrique, et mémorisation directionnelle court-terme (mémoire géométrique).
- **L'ingénierie système sous ROS 2** : Maîtrise de l'architecture distribuée (multi-nœuds), communication asynchrone par *Topics*, interaction synchrone/asynchrone par *Services* pour la reconfiguration dynamique de capteurs (paramètres caméra), et diagnostic système via `rqt_graph`.

Structure du Travail Demandé

Le document est organisé en trois exercices progressifs :

- **Exercice 1 (Vision)** : Analyse mathématique et géométrique de l'algorithme de perception OpenCV, de l'immunité anti-reflet et de la gestion des modes dégradés.
- **Exercice 2 (Commande)** : Modélisation du comportement cinématique du robot, rôle du terme dérivé et de la machine à états face aux virages serrés.
- **Exercice 3 (Intégration)** : Cartographie complète des flux ROS 2, étude du protocole de « secours solaire » par requêtes de services et implémentation finale de la boucle algorithmique manquante.

1 Exercice 1 : Traitement d'Images et Perception de l'Environnement (Vision)

1.1 Idée générale

Pour se déplacer, le robot s'appuie désormais sur un capteur monoculaire (caméra couleur). Le nœud `VisionLineNode` traite le flux vidéo brut à l'aide de la bibliothèque OpenCV pour isoler une piste matérialisée par des lignes jaunes. Afin d'assurer un suivi robuste malgré les perturbations lumineuses, le script utilise un filtrage de persistance temporelle, segmente l'espace en trois horizons géométriques distinctes ($L1, L2, L3$), et applique un filtre de validation géométrique adaptatif basé sur l'écartement théorique des lignes (immunité anti-reflets).

FIGURE 1 – Représentation géométrique des horizons d'analyse ($L1, L2, L3$) et fenêtrage de la route dans le plan image.

1.2 Travail demandé

1. **Segmentation Couleur (HSV)** : Analysez la méthode `image_callback()`. Pourquoi le script convertit-il l'image du format BGR vers le format HSV avant d'appliquer la fonction `cv2.inRange()` ? Quel est l'avantage concret de l'espace HSV par rapport à l'espace RVB/BGR classique lors d'une application de robotique mobile en conditions réelles ?
2. **Filtre de Persistance Temporelle** : Observez l'implémentation de la file d'attente circulaire `self.historique_masques` (`deque`).
 - Quel calcul mathématique/matriciel est opéré via `np.sum(... == 255, axis=0)` ?
 - Quel est le rôle de ce mécanisme ? Expliquez en quoi il immunise le robot contre les phénomènes de « clignotement » (perte furtive d'une ligne sur une seule trame d'image).
3. **Analyse Multilevel et Filtrage Géométrique** : Le code définit trois lignes d'horizon virtuelles ($Y_{L1} = 450, Y_{L2} = 320, Y_{L3} = 170$).
 - Pourquoi la valeur de largeur mémorisée dans `self.largeurs_memoire` diminue-t-elle à mesure que l'on s'élève dans l'image ($L1 \rightarrow L2 \rightarrow L3$) ? Quel effet d'optique est ici pris en compte ?

- Expliquez le fonctionnement de la méthode `filtrer_paire_par_largeur()`. Comment élimine-t-elle un reflet parasite jaune (par exemple sur un sol brillant) situé à côté de la vraie piste ?
 - Décrivez l'intérêt d'avoir implémenté un filtre passe-bas sur la mise à jour de la largeur cible : $0.9 * \text{largeur_cible} + 0.1 * \text{largeur_calculee}$.
4. **Modes Dégradés (Suivi Mono-ligne) :** Si le robot négocie un virage très serré et que la ligne gauche sort complètement du champ de vision de la caméra, comment le script parvient-il à calculer une estimation de l'erreur par rapport au centre du couloir (Condition E du script) ?

2 Exercice 2 : Contrôle par Correcteur PD et Gestion Événementielle

2.1 Idée générale

Une fois l'erreur géométrique calculée par le nœud de vision, elle est transmise au nœud de commande `RobotSimple`. Ce nœud implémente un régulateur **Proportionnel-Dérivé (PD)** pour générer la vitesse de rotation. Il intègre également une machine à états sophistiquée chargée d'adapter la cinématique du robot selon le contexte (`NOMINAL`, `EPINGLE`, `BIFURCATION`, `DETRESSE`) ainsi qu'un service d'adaptation dynamique de la caméra face aux variations d'ensoleillement (« secours solaire »).

2.2 Travail demandé

1. **Modélisation du Correcteur PD :** Observez le bloc de calcul des variables P et D .
 - Pourquoi l'équation de la vitesse angulaire finale est-elle affectée d'un signe négatif (`angular_z_pid = -(P + D)`) ? À quel concept physique/automatique cela correspond-il ?
 - Quelle information essentielle apporte le terme dérivé (D) par rapport au terme proportionnel (P) lorsque le robot aborde l'entrée d'une ligne droite à grande vitesse ?
2. **Profil de Vitesse Dynamique :** Dans le mode `NOMINAL`, observez la formulation de la vitesse linéaire `msg.linear.x`. Expliquez comment le robot adapte sa vitesse par rapport à son alignement. Quel est l'intérêt de cette approche pour la préservation mécanique du matériel ?
3. **Mémoire Géométrique :** Étudiez le calcul de la variable `self.angle_accumule` ainsi que la gestion de `self.dernier_sens_virage`. Comment cette mémoire court-terme est-elle exploitée par le robot s'il entre dans l'état de `DETRESSE` (piste totalement perdue) ? En quoi cela empêche-t-il le robot de partir dans la mauvaise direction ?

3 Exercice 3 : Intégration Système et Communication Inter-Nœuds

3.1 Idée générale

Dans cette ultime partie, nous étudions l'interaction globale des deux programmes au sein de l'écosystème ROS 2. La perception et l'action s'exécutent en parallèle, s'échangeant des informations standardisées à des fréquences différentes par le biais de messages asynchrones et de requêtes de services.

FIGURE 2 – Architecture logicielle, interconnexions par Topics et Services entre la caméra, le nœud de vision et le nœud de commande cinématique.

3.2 Travail demandé

1. **Cartographie des flux ROS 2** : Dressez un tableau récapitulant l'ensemble des communications mis en œuvre entre le nœud `vision_line_node` et le nœud `control_line_node`. Le tableau devra faire apparaître : le nom du topic/service, le type de message ROS 2 associé, et le rôle de cette donnée.
2. **Analyse de service (« Secours Solaire »)** : Le robot intègre un client configuré sur le service `/camera/camera/set_parameters`.
 - Quelle est la différence fondamentale entre l'envoi d'une consigne de vitesse via le topic `/cmd_vel` et la modification de l'exposition de la caméra via ce service ? Pourquoi utilise-t-on un **Service** plutôt qu'un **Topic** ici ?
 - Pourquoi l'appel est-il réalisé de manière asynchrone (`call_async()`) plutôt que bloquante ?
3. **À compléter dans le code** : Ouvrez votre script de contrôle. La structure algorithmique centrale de la méthode `boucle()` déterminant les comportements en fonction des modes a été retirée. En exploitant les variables d'états calculées en amont (`angular_z_pid`, `l1_ok`, `l3_ok`, `self.erreur`, etc.), complétez les zones manquantes pour restaurer le suivi de ligne complet du véhicule.